
Improving Model Performance with Efficient Hyperparameter Tuning: The Mathematics Behind the TPE Algorithm (Part 2 of 3)

Dr. Simone Giorgi
Dublin, Ireland

July 20, 2026

1 Introduction

In the previous article “Improving Model Performance with Efficient Hyperparameter Tuning: Introduction to the Tree-structured Parzen Estimator (TPE) [Part 1 of 3]”. I introduced the fundamentals of hyperparameter optimization and discussed why Bayesian optimization methods have become an attractive alternative to traditional approaches such as Grid Search and Random Search. I also presented the basic intuition behind the Tree-structured Parzen Estimator (TPE) [2] [4], the Bayesian optimization algorithm used by default in Optuna [3] [1], explaining how it learns from previous evaluations to progressively focus the search on promising regions of the hyperparameter space instead of exploring candidate configurations blindly. In this article, we move beyond the high-level intuition and examine the mathematical principles that make TPE work.

Rather than directly modeling the objective function, as done by Gaussian Process-based Bayesian optimization, TPE models two probability density functions over the hyperparameter space: one representing configurations that have produced good validation losses and another representing those associated with poor performance. These probability models are then used to estimate which candidate hyperparameter vectors are most likely to improve the current best solution. We will derive the definitions of these probability densities, explain how they are constructed from previous trials using kernel density estimation, and show why maximizing their ratio naturally guides the optimization process toward increasingly better hyperparameter configurations. By the end of this article, the mathematical foundations of TPE should become much clearer, making it easier to understand both the original algorithm and its implementation in modern frameworks such as Optuna.

2 Notation

$\mathcal{D} = \{(\lambda_i, e_{\lambda_i})\}_{i=1}^N$: Sample set of hyperparameter vectors and corresponding validation errors.

$\mathcal{D}^{(\ell)} = \{(\lambda_i^{(\ell)}, e_{\lambda_i^{(\ell)}})\}_{i=1}^{N^{(\ell)}}$: Set of good hyperparameter samples.

$\mathcal{D}^{(g)} = \{(\lambda_i^{(g)}, e_{\lambda_i^{(g)}})\}_{i=1}^{N^{(g)}}$: Set of bad hyperparameter samples.

e_{λ} : Validation error obtained after training a model with hyperparameters λ .

$g(\lambda)$: Parzen estimator formed from the bad hyperparameter samples.

$\mathcal{K}_{\Sigma_i}(\lambda - \lambda_i)$: Gaussian kernel centered at λ_i with covariance matrix Σ_i .

$\ell(\lambda)$: Parzen estimator formed from the good hyperparameter samples.

m : Number of random evaluations during the initialisation.

M : Number of promising hyperparameter vectors extracted by using $\ell(\lambda)$.

N : Number of hyperparameter samples.

$N^{(\ell)}$: Number of good hyperparameter samples.

$N^{(g)}$: Number of bad hyperparameter samples.

λ : Generic hyperparameter vector.

$\lambda_i^{(\ell)}$: Hyperparameter vector classified as good.

$\lambda_i^{(g)}$: Hyperparameter vector classified as bad.

$\tilde{\lambda}_i$: Candidate good hyperparameter vector sampled from the good distribution.

Σ : Kernel covariance matrix.

σ_i^2 : Variance of the i -th hyperparameter.

3 Kernels

The kernels are one of the central components of the TPE algorithm. Rather than fitting a single global probabilistic model, TPE represents the distribution of good and bad hyperparameter configurations using kernel density estimation (KDE). Each previously evaluated hyperparameter configuration contributes one kernel, and the sum of these kernels forms a smooth estimate of the probability densities $\ell(\lambda)$ and $g(\lambda)$, where $\lambda \in \mathbb{R}^d$ is the hyperparameter vector.

In KDE, the choice of kernel determines the shape of the smooth function placed around each observation. In principle, any probability density function can be used as a kernel, but a very common option is the Gaussian kernel (also called Parzen kernels):

$$\mathcal{K}_{\Sigma}(\lambda - \mu) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\lambda - \mu)^T \Sigma^{-1}(\lambda - \mu)\right)$$

where:

- d is the dimension of the hyperparameter vector,
- $\mu \in \mathbb{R}^d$ is the mean vector (the center of the kernel):

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_d \end{bmatrix}$$

- $\Sigma \in \mathbb{R}^{d \times d}$ is the covariance matrix;

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1d} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{d1} & \sigma_{d2} & \cdots & \sigma_d^2 \end{bmatrix}$$

- where diagonal entries are the variances:

$$\Sigma_{ii} = \sigma_i^2$$

- while the off-diagonal entries are the covariances:

$$\Sigma_{ij} = \text{Cov}(\lambda_i, \lambda_j)$$

- $|\Sigma|$ denotes the determinant of the covariance matrix.

Figure 1 shows the contour plot of a generic two-dimensional Gaussian kernel ($d = 2$), with $\boldsymbol{\mu} = (0, 0)$ and a non-diagonal covariance matrix $\boldsymbol{\Sigma}$. Since the marginal variances are different, $\sigma_1^2 \neq \sigma_2^2$, the distribution exhibits an elliptical shape. Moreover, the off-diagonal covariance terms introduce a correlation between the two dimensions, causing the principal axes of the ellipse to be rotated with respect to the coordinate axes.

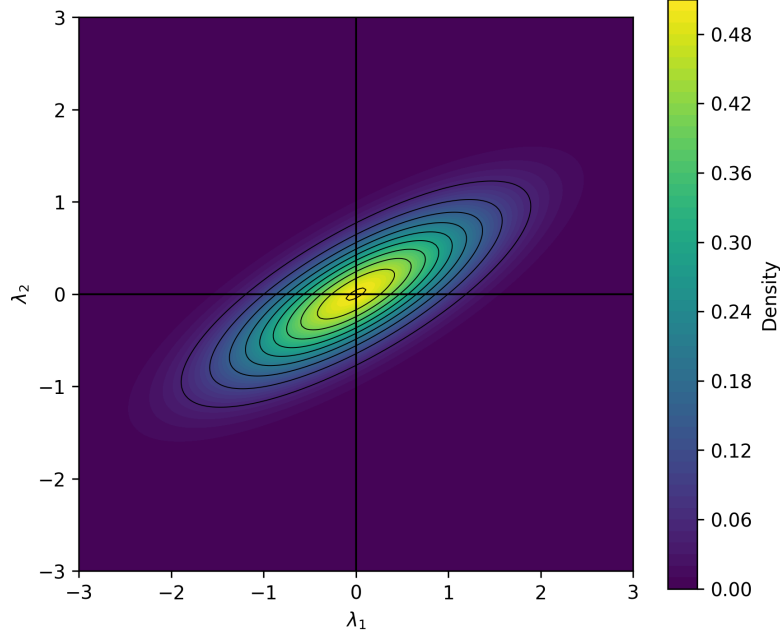


Figure 1: Contour plot of a generic two-dimensional Gaussian kernel ($d = 2$) with $\boldsymbol{\mu} = (0, 0)$ and a non-diagonal covariance matrix $\boldsymbol{\Sigma}$. The different marginal variances ($\sigma_1^2 \neq \sigma_2^2$) and covariance between dimensions produce rotated elliptical contours.

3.1 Special case: independent hyperparameters

If the variables are independent, then the covariance matrix is diagonal,

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_d^2 \end{bmatrix}$$

and the multivariate Gaussian density simplifies to

$$\mathcal{K}_{\boldsymbol{\Sigma}}(\boldsymbol{\lambda} - \boldsymbol{\mu}) = \prod_{i=1}^d \frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(-\frac{(\lambda_i - \mu_i)^2}{2\sigma_i^2}\right)$$

Thus, the multivariate Gaussian distribution factorizes into the product of d independent one-dimensional Gaussian distributions. In the context of TPE, the standard deviations σ_i are called bandwidths.

Geometrically, this corresponds to Gaussian kernels whose elliptical contours are aligned with the coordinate axes, with the principal axes of the distribution parallel to the original coordinate directions.

In particular, Figure 2 shows the case of a Gaussian kernel with a diagonal covariance matrix having identical diagonal elements. In this case, the resulting distribution is isotropic, producing circular contours.

Figure 3, instead, illustrates the case of a Gaussian kernel with a diagonal covariance matrix having different diagonal elements. The unequal variances along the two dimensions produce elliptical contours, while the principal axes remain aligned with the coordinate axes.

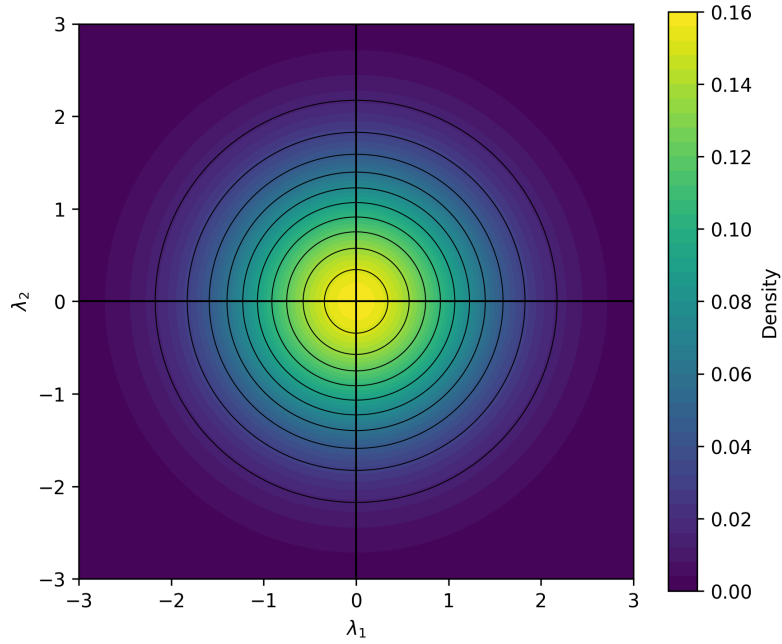


Figure 2: Contour plot of an isotropic two-dimensional Gaussian kernel with a diagonal covariance matrix Σ having identical diagonal elements. The resulting circular contours indicate equal variance in all directions.

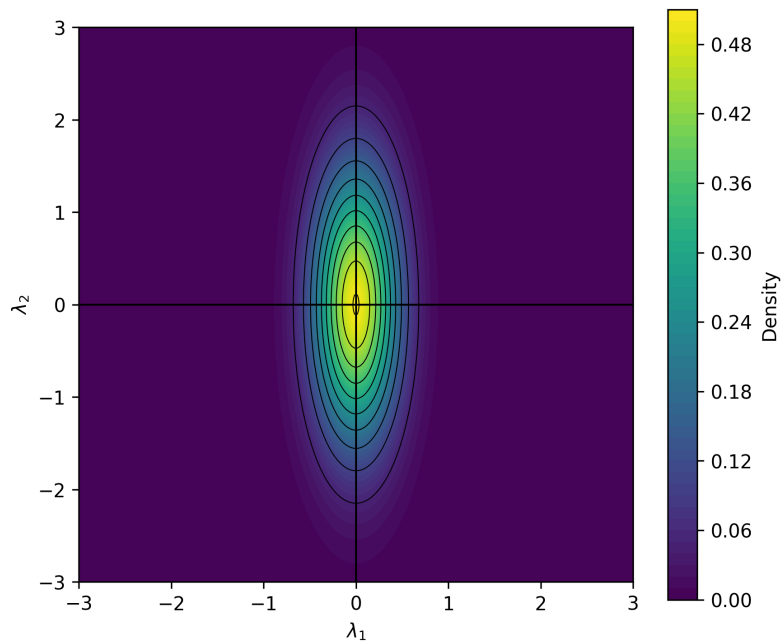


Figure 3: Contour plot of a two-dimensional Gaussian kernel ($d = 2$) with $\mu = (0,0)$ and a diagonal covariance matrix Σ , where $\sigma_1^2 \neq \sigma_2^2$. The different variances produce elliptical contours aligned with the coordinate axes.

3.2 Kernel covariance matrix (bandwidths)

In Section 3.1, it has been explained the case of kernels having diagonal correlation matrix. In practice, this is a common implementation assumption, since a full covariance matrix would require estimating $N(N+1)/2$ parameters for every Gaussian kernel. For example, having $N = 20$ already requires $20 \cdot 21/2 = 210$ covariance parameters per kernel. Since Bayesian optimization usually has relatively few evaluated hyperparameter vectors, estimating such large covariance matrices would be:

- statistically unreliable,
- computationally expensive,
- numerically unstable.

Using diagonal covariance matrices makes TPE considerably more robust in high-dimensional optimization problems.

The use of diagonal covariance matrices means that:

- each hyperparameter has its own independent variance,
- Gaussian kernels may stretch differently along each coordinate axis,
- Gaussian kernels cannot rotate,
- individual kernels cannot explicitly model correlations between hyperparameters.

This simplification is one of the reasons why TPE remains computationally efficient even for optimization problems involving many hyperparameters.

Section 4.3.1 provides further details on the implementation of the kernel bandwidth calculation.

4 TPE Algorithm steps

This section describes the main steps of the TPE algorithm (see Figure 4).

After an initialisation phase, in which a first set of hyperparameter configurations is randomly sampled and evaluated, the algorithm proceeds iteratively. At each iteration, the available observations are first used to partition the dataset into two subsets:

$$\mathcal{D}^{(\ell)} \quad \text{and} \quad \mathcal{D}^{(g)},$$

corresponding respectively to the configurations associated with better and worse objective function values. These two subsets are then used to construct the Parzen estimators:

$$\ell(\boldsymbol{\lambda}) \quad \text{and} \quad g(\boldsymbol{\lambda}),$$

using kernel density estimation, where each density is represented as a sum of kernels centred at the previously evaluated hyperparameter configurations.

The ratio $\ell(\boldsymbol{\lambda})/g(\boldsymbol{\lambda})$ is then used as a surrogate acquisition criterion to identify promising hyperparameter configurations. A new candidate configuration is selected, evaluated by training the ML/DL model, and its corresponding objective value is added to the set of observations. The updated dataset is then used in the following iteration, progressively improving the efficiency of the hyperparameter search.

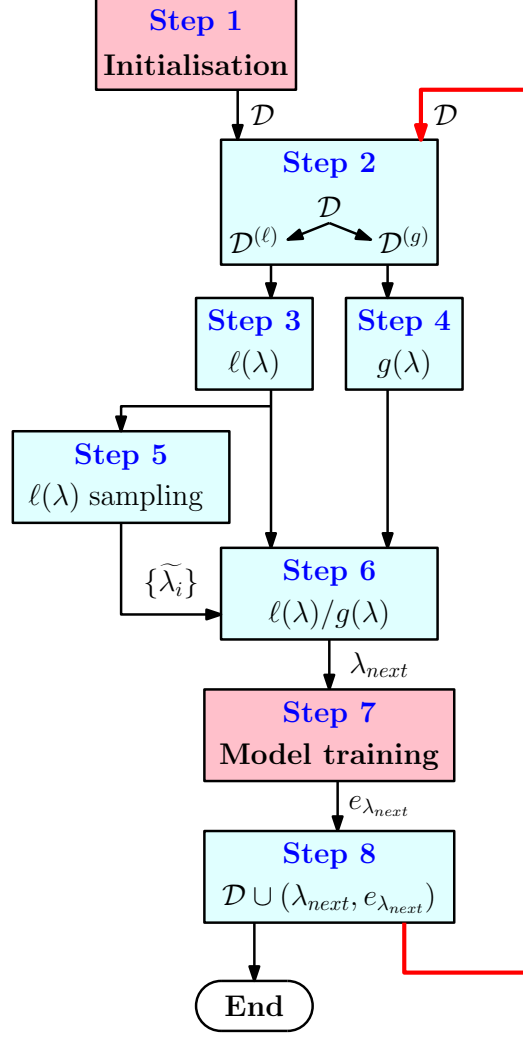


Figure 4: Overview of the TPE algorithm workflow. In Steps 1 and 7, the ML/DL model is trained and validated to evaluate the performance of the selected hyperparameter configurations; these steps are highlighted in red to emphasize their high computational cost, as they require the expensive execution of the model training process.

4.1 Step 1: Initialisation – First m random evaluations

The TPE algorithm begins with an initial exploration phase in which no probabilistic model of the objective function is yet available. Consequently, the first m hyperparameter vectors are sampled independently from the prior distribution over the search space, $p(\lambda)$. This distribution is defined by the user through the specification of each hyperparameter (uniform, log-uniform, categorical, integer, etc.). The initial set of sampled hyperparameter vectors is therefore:

$$\{\lambda_1, \lambda_2, \dots, \lambda_m\}.$$

Each hyperparameter vector is then used to train the ML/DP model, producing the corresponding validation error:

$$\{e_{\lambda_1}, e_{\lambda_2}, \dots, e_{\lambda_m}\}.$$

At the end of this initialisation phase, the algorithm has collected the first observations

$$\mathcal{D} = \{(\lambda_i, e_{\lambda_i})\}_{i=1}^m,$$

which constitute the initial dataset used to construct the two kernel density estimators, $\ell(\boldsymbol{\lambda})$ and $g(\boldsymbol{\lambda})$, during the next iteration. From this point onward, the algorithm switches from random exploration to model-based Bayesian optimisation.

4.2 Step 2: Partitioning the observations into good and bad sets

This step marks the beginning of the TPE optimization loop, which uses the observations obtained in the previous step:

$$\mathcal{D} = \{(\boldsymbol{\lambda}_i, e_{\boldsymbol{\lambda}_i})\}_{i=1}^N$$

A threshold value e^* is selected as the γ -quantile of the observed validation errors (in practice, $\gamma \approx 0.1 - 0.2$ is commonly used). The observations are therefore partitioned into two disjoint datasets, the set of “good” observations:

$$\mathcal{D}^{(\ell)} = \{(\boldsymbol{\lambda}_i, e_{\boldsymbol{\lambda}_i}) : e_{\boldsymbol{\lambda}_i} < e^*\},$$

and the set of “bad” observations:

$$\mathcal{D}^{(g)} = \{(\boldsymbol{\lambda}_i, e_{\boldsymbol{\lambda}_i}) : e_{\boldsymbol{\lambda}_i} \geq e^*\}.$$

It is important to note that this partition is *not permanent*. At each iteration, the newly evaluated observation is added to the dataset, the threshold e^* is recomputed using all available observations, and the datasets $\mathcal{D}^{(\ell)}$ and $\mathcal{D}^{(g)}$ are reconstructed accordingly. Consequently, a hyperparameter vector that was previously classified as “good” may later be reclassified as “bad”, and vice versa, as more information about the objective function becomes available.

4.3 Step 3: Calculation of $\ell(\boldsymbol{\lambda})$

$\ell(\boldsymbol{\lambda}) : \mathbb{R}^d \rightarrow \mathbb{R}$ is a multivariate probability density functions that describe where good hyperparameter configurations are located. It is constructed from the available samples $\mathcal{D}^{(\ell)}$ and it is expressed as summation of kernels (see Section 3):

$$\ell(\boldsymbol{\lambda}) = \frac{1}{N^{(\ell)}} \sum_{i=1}^{N^{(\ell)}} \mathcal{K}_{\Sigma_i}(\boldsymbol{\lambda} - \boldsymbol{\lambda}_i^{(\ell)})$$

Each i -th kernel is centered in the corresponding $\boldsymbol{\lambda}_i^{(\ell)}$ and has bandwidth specified by Σ_i .

The use of bell-shaped kernels implicitly assumes that $\ell(\boldsymbol{\lambda})$ is locally smooth, so that nearby hyperparameter configurations tend to have similar performance. As a consequence, the kernel density estimators naturally form regions of high density corresponding to groups of good configurations. The bell-shaped kernels encode this assumption by assigning high probability to points close to previously observed configurations and progressively lower probability to more distant points. Because each kernel assigns its highest density to points close to its center, the KDE implicitly assumes that nearby hyperparameter configurations are likely to exhibit similar performance.

The effectiveness of TPE relies on the assumption that the objective function is reasonably smooth in the hyperparameter space. If two very similar hyperparameter vectors can produce completely different objective values, then the KDE approximation becomes less informative. Fortunately, for many machine learning hyperparameter optimization problems, this smoothness assumption is approximately satisfied. Small changes in continuous hyperparameters (e.g., learning rate, regularization strength, or dropout rate) usually produce relatively small changes in model performance.

4.3.1 Bandwidth calculation

As discussed in Section 3.2, TPE implementations commonly use diagonal covariance matrices for the Gaussian kernels. The bandwidths are estimated using a data-adaptive heuristic based on the local spacing between neighboring samples.

In particular, for each kernel centered at an observed hyperparameter vector λ_i , the bandwidth is determined from the local distribution of samples:

$$\sigma_i = f(\text{local neighborhood of } \lambda_i).$$

Consequently, densely sampled regions lead to smaller bandwidths (narrow kernels), whereas sparse regions lead to larger bandwidths (wider kernels). This adaptive procedure allows the Parzen estimator to adjust its smoothness according to the observed hyperparameter distribution.

In implementations such as Optuna, the bandwidths are recomputed every time the density estimators $\ell(\lambda)$ and $g(\lambda)$ are reconstructed from the updated observations. Therefore, while the kernel centers λ_i remain fixed once evaluated, their associated bandwidths σ_i may change as new samples are collected. Hence, TPE should be viewed as an iteratively reconstructed Parzen estimator, where both the sample set and the kernel smoothing parameters evolve during the optimization process.

4.4 Step 4: Calculation of $g(\lambda)$

Similarly to $\ell(\lambda)$, introduced in Section 4.3, $g(\lambda) : \mathbb{R}^d \rightarrow \mathbb{R}$ is a multivariate pdf that models the distribution of the *bad* hyperparameter configurations observed during the optimization process, and it is constructed from the set of bad samples $\mathcal{D}^{(g)}$:

$$g(\lambda) = \frac{1}{N^{(g)}} \sum_{i=1}^{N^{(g)}} \mathcal{K}_{\sigma_i}(\lambda - \lambda_i^{(g)}).$$

Each kernel contributes a smooth local estimate of the density around one bad hyperparameter configuration. Their sum produces a continuous approximation of the probability density of observing bad configurations throughout the hyperparameter space.

Although $\ell(\lambda)$ and $g(\lambda)$ describe the regions where good and bad hyperparameter configurations are concentrated, they do *not* represent the probabilities that a given hyperparameter vector is good or bad. Consequently,

$$\ell(\lambda) + g(\lambda) \neq 1.$$

Instead, they provide the probability to find good and bad hyperparameters, once integrated on a specific area of the hyperparameter space.

Instead, they define pdf whose integrals over a given region of the hyperparameter space yield the probability of finding good or bad hyperparameter configurations within that region.

4.5 Step 5: Sampling M candidate hyperparameter vectors from $\ell(\lambda)$

The probability density function $\ell(\lambda)$ models the region of the hyperparameter space where good configurations have previously been observed. Consequently, it is used to generate M promising candidate hyperparameter vectors that may be selected as the next configuration to evaluate:

$$\{\tilde{\lambda}_1, \tilde{\lambda}_2, \dots, \tilde{\lambda}_M\} \quad (1)$$

Notice that the density $g(\lambda)$ is *not* used during this sampling stage. Its role is only to evaluate the sampled candidates afterwards through the acquisition criterion $\ell(\lambda)/g(\lambda)$.

Several sampling strategies can be adopted. For example, Optuna generates the M candidate vectors according to the following procedure.

Assume that the density $\ell(\lambda)$ is represented by the summation of $N^{(\ell)}$ Gaussian kernels.

1. Uniformly select one of the $N^{(\ell)}$ Gaussian kernels. The selection is performed with replacement, meaning that the same kernel may be selected multiple times.
2. Sample randomly one hyperparameter vector $\tilde{\lambda}$ from the selected Gaussian kernel.
3. Repeat Steps 1. and 2. until M candidate vectors have been generated.

As a consequence, the generated candidates are distributed according to $\ell(\lambda)$ and are concentrated in regions of the search space that are more likely to contain high-performing hyperparameter configurations.

4.5.1 Why $\ell(\lambda)$ sampling is used

Solving directly the following problem is difficult:

$$\lambda_{\text{next}} = \arg \max_{\lambda} \frac{\ell(\lambda)}{g(\lambda)}$$

Instead, the $\ell(\lambda)$ sampling makes the identification of λ_{next}

- computationally efficient,
- robust,
- derivative-free.

4.6 Step 6: Selection of the next hyperparameter configuration using $\ell(\lambda)/g(\lambda)$

After generating the M candidates $\tilde{\lambda}_i$ in Eq. (1), each candidate is evaluated to determine which one should be selected as the next hyperparameter configuration, λ_{next} .

A desirable candidate should satisfy the following two properties:

- it should have a large value of the distribution of successful configurations $\ell(\lambda)$,
- it should have a small value of the distribution of unsuccessful configurations $g(\lambda)$.

Therefore, TPE selects the candidate that maximizes the ratio:

$$\lambda_{\text{next}} = \arg \max_{\tilde{\lambda}_i} \frac{\ell(\tilde{\lambda}_i)}{g(\tilde{\lambda}_i)}, \quad (2)$$

naturally favoring candidates that are simultaneously close to previously successful configurations and far from previously unsuccessful ones.

It is important to underline that this selection step is performed without evaluating the objective function e_{λ} , which would require a complete training of the machine learning model. Therefore, TPE avoids the computationally expensive model evaluation for most candidate configurations and uses only the surrogate distributions $\ell(\lambda)$ and $g(\lambda)$ to identify the most promising hyperparameter vector.

4.7 Step 7: Model training

Sampling candidates from $\ell(\lambda)$ and evaluating them with Eq. (2) does not guarantee that the selected hyperparameter vector λ_{next} is actually good. Rather, it reflects the current belief that the candidate is likely to perform well because it lies in a region where good configurations have been observed in the past. The true quality of the candidate is revealed only after training and evaluating the model (the calculation of $e_{\lambda_{\text{next}}}$).

4.8 Step 8: $\mathcal{D}$ update

The new $(\lambda_{\text{next}}, e_{\lambda_{\text{next}}})$ is incorporated into the available dataset:

$$\mathcal{D} \cup (\lambda_{\text{next}}, e_{\lambda_{\text{next}}}) \longrightarrow \mathcal{D}$$

Once $\mathcal{D}$ has been updated, the loop proceeds to the next iteration, where the updated observations are used to reconstruct $\mathcal{D}^{(\ell)}$, $\mathcal{D}^{(g)}$, $\ell(\lambda)$ and $g(\lambda)$.

Bibliography

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.
- [2] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyperparameter optimization. *Advances in neural information processing systems*, 24, 2011.
- [3] Optuna. Optuna documentation. URL <https://optuna.org/>.
- [4] Shuhei Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. 2023. URL <https://arxiv.org/abs/2304.11127>.